



**SAKARYA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ**

SİSTEM PROGRAMLAMA

3.PROJE

Öğrencilerin

Adı Soyadı : Umutcan Yıldız, Enes Doğru, Nihat Can Akkaya, Süreyya Sönmez

Okul No : 23010903072, 23010903048, 22010903092, 23010903078



İçindekiler

1. GİRİŞ VE LİTERATÜR ARAŞTIRMASI.....	4
1.1. Gömülü Sistemlerde Program Yükleme Mimarileri.....	4
1.2. Seri Haberleşme ve Veri Doğrulama Protokolleri.....	4
1.2.1. Checksum (Toplam Değer Kontrolü)	4
1.2.2. CRC (Döngüsel Karmaşıklık Denetimi)	5
2. SİTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI.....	5
2.1. Toolchain Arayüz Standartları	5
2.2. FPGA Loader ve PicoRV32 Bellek Haritası (Memory Map)	7
3. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ.....	9
3.1. Deney Tasarımı ve Test Senaryoları	9
3.2. Veri Toplama ve Donanım Metrikleri	12
4. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ.....	13
4.1. Sürdürülebilirlik ve Yeşil Bilişim (Green Computing)	13
4.2. Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık	13
4.3. Fonksiyonel Güvenlik ve Sağlık.....	14
4.4. E-Atık Yönetimi ve Döngüsel Ekonomi.....	14
5. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI.....	15
6. BİREYSEL KATKI BEYANI	16
7. KAYNAKÇA.....	16

ŞEKİL LİSTESİ

Şekil 1 Sistem Toolchain Şeması.....	6
Şekil 2 Bellek Haritası.....	7
Şekil 3 Loader FSM Şeması.....	8
Şekil 4 Test Senaryoları Gerçekleşme Süreleri.....	12
Şekil 5 Gowin EDA Enerji İstatistikleri.....	13
Şekil 6 SKA Tablosu.....	14
Şekil 7 Takım Çalışması.....	15

TABLO LİSTESİ

Tablo 1 Checksum ve CRC Karşılaştırma Tablosu.....	5
--	---

1. GİRİŞ VE LİTERATÜR ARAŞTIRMASI

1.1. Gömülü Sistemlerde Program Yükleme Mimarileri

Mikroişlemci mimarilerinde tasarlanan uygulamaların hedef donanım üzerindeki kalıcı veya geçici belleklere (Block RAM/SRAM) fiziksel katman üzerinden aktarılması, sistem araç zincirinin en kritik aşamalarından biridir. Literatürde gömülü sistem yazılımlarının güncellenmesi ve çalıştırılması amacıyla yaygın olarak kullanılan üç temel bootloader/loader mimarisi bulunmaktadır: UART, SPI ve JTAG [1].

UART (Universal Asynchronous Receiver-Transmitter): İki cihaz arasında ortak bir zamanlama sinyaline gerek duymadan, sadece biri veri alma diğeri veri gönderme olmak üzere iki hat üzerinden doğrudan haberleşme sağlar. FPGA üzerinde çok az donanım alanı kaplar. Cihazlar arasında karmaşık bir yönetim hiyerarşisi kurmaya gerek bırakmadığı için, yazılım yükleme sistemlerinde kolayca kontrol mekanizmaları kurulmasına imkan tanır.

SPI (Serial Peripheral Interface): İki cihazın aynı zamanlama sinyaliyle tam uyumlu ve çift yönlü çalıştığı bir mimaridir. Yüksek hızlara çıkabildiği için büyük boyutlu dosyaların aktarımında oldukça başarılıdır. Ancak cihazların birbiriyle haberleşebilmesi ve veri akışını yönetebilmesi için en az 4 fiziksel kabloya (veri giriş, veri çıkış, saat sinyali ve cihaz seçimi) ihtiyaç duyar. Bu durum, FPGA üzerindeki bağlantı yollarını ve fiziksel pin kaynaklarını gereksiz yere tüketir.

JTAG (Joint Test Action Group): Doğrudan entegre devrelerin iç katmanlarına ve işlemci registerlarına (hafıza hücrelerine) erişim sağlayan donanımsal bir hata ayıklama ve programlama arayüzüdür. Sisteme tam bir kontrol gücü sunsa da, kendi içindeki kontrol mekanizması çok karmaşıktır ve işlemciye ek yük getirir. Bu yüzden, sadece tasarlanan bir programı yükleyip çalıştırmak isteyen hafif yazılımsal yükleyiciler için aşırı donanım harcanmasına neden olur.

Bu projede yazılım tabanlı bir UART yükleyici (loader) sistemi tercih edilmiştir. Bu seçimin temel nedenleri; Tang Nano 9K çipinin kapasite sınırlarına uygun olması, en az sayıda fiziksel bağlantı bacağına (pin) ihtiyaç duyması ve PicoRV32 işlemcisinin araya başka karmaşık donanımlar girmeden kendi hafızasına kolayca, doğrudan veri yazabilmesidir.

1.2. Seri Haberleşme ve Veri Doğrulama Protokolleri

Bilgisayardan çipe kablo üzerinden komut gönderirken, kablodaki anlık güç dalgalanmaları veya bağlantıdaki ufak pürüzler sebebiyle gönderdiğimiz 1 ve 0'lar yolda değişime uğrayabilir. İşlemciye ulaşan mesajdaki tek bir 1'in 0'a dönüşmesi bile, işlemcinin o komutu tamamen yanlış anlamasına ve sistemin kilitlenip çökmesine neden olur. Bu tehlikeyi önlemek ve mesajın karşıya eksiksiz ulaştığını matematiksel olarak doğrulamak için sistemlerde Checksum (Toplam Değer Kontrolü) ve CRC gibi güvenlik yöntemleri tercih edilir [2].

1.2.1. Checksum (Toplam Değer Kontrolü)

Gönderilecek olan tüm verilerin (baytların) matematiksel olarak alt alta toplanması ve elde edilen bu toplam değer paketin en sonuna eklenerek karşıya iletilmesidir. Alıcı cihaz, gelen verileri kendi içinde tekrar toplar ve çıkan sonucu en sondaki toplam değerle karşılaştırır. İşlemci üzerinde neredeyse hiç yer kaplamaz çünkü sadece basit bir toplama işlemi yapar. Giden mesajda tek bir karakter bile bozulsa bunu anında fark eder; ancak verideki birden fazla bozulma birbirini matematiksel olarak dengelerse (biri artarken diğeri azalırsa) hatayı gözden kaçırma ihtimali vardır.

1.2.2. CRC (Döngüsel Karmaşıklık Denetimi)

Basit bir toplama işlemi yapmak yerine, gönderilecek verileri çok daha karmaşık bir matematiksel bölme işlemine (polinom bölmesi) sokarak kalan sayıyı şifreye dönüştüren gelişmiş bir yöntemdir [3]. Peş peşe gelen kopmalarda veya verinin büyük bir kısmının bozulduğu durumlarda bile hatayı yakalama oranı yüksektir. Ancak bu karmaşık matematiği işlemcinin içinde anlık olarak hesaplayabilmek için çok daha fazla elektronik parçaya ve devreye ihtiyaç duyar; bu da çip üzerinde gereksiz yer kaplamasına (donanım maliyetinin artmasına) neden olur (PÇ6).

Tablo 1 Checksum ve CRC Karşılaştırma Tablosu

Karşılaştırma Ölçütü	Checksum (Toplam Değer Kontrolü)	CRC (Döngüsel Karmaşıklık Denetimi)
Hesaplama Mantığı	Gönderilen verilerin matematiksel olarak alt alta toplanmasıdır.	Verilerin karmaşık bir matematiksel bölme işlemine (polinom bölmesi) sokulmasıdır.
Çip Üzerindeki Yüğü (Donanım)	Çok Düşük (Sadece basit bir toplama ünitesi gerektirir).	Yüksek (Çip üzerinde çok daha fazla elektronik devre ve parça kullanılır).
Hata Yakalama Başarısı	Tekli hataları anında bulur; ancak zıt hatalar birbirini matematiksel olarak dengelerse gözden kaçırabilir.	Peş peşe gelen veya büyük çaplı veri bozulmalarında bile hata yakalama oranı yüksektir.

2. SİTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI

2.1. Toolchain Arayüz Standartları

Şekil 1'de detaylıca görüldüğü gibi, geliştirdiğimiz bu sistem; bilgisayarda yazdığımız bir kodun çip (FPGA) üzerinde fiziksel bir harekete dönüşmesine kadar geçen tüm süreci köprü gibi birbirine bağlar. Bu akış üç temel aşamadan oluşmaktadır:

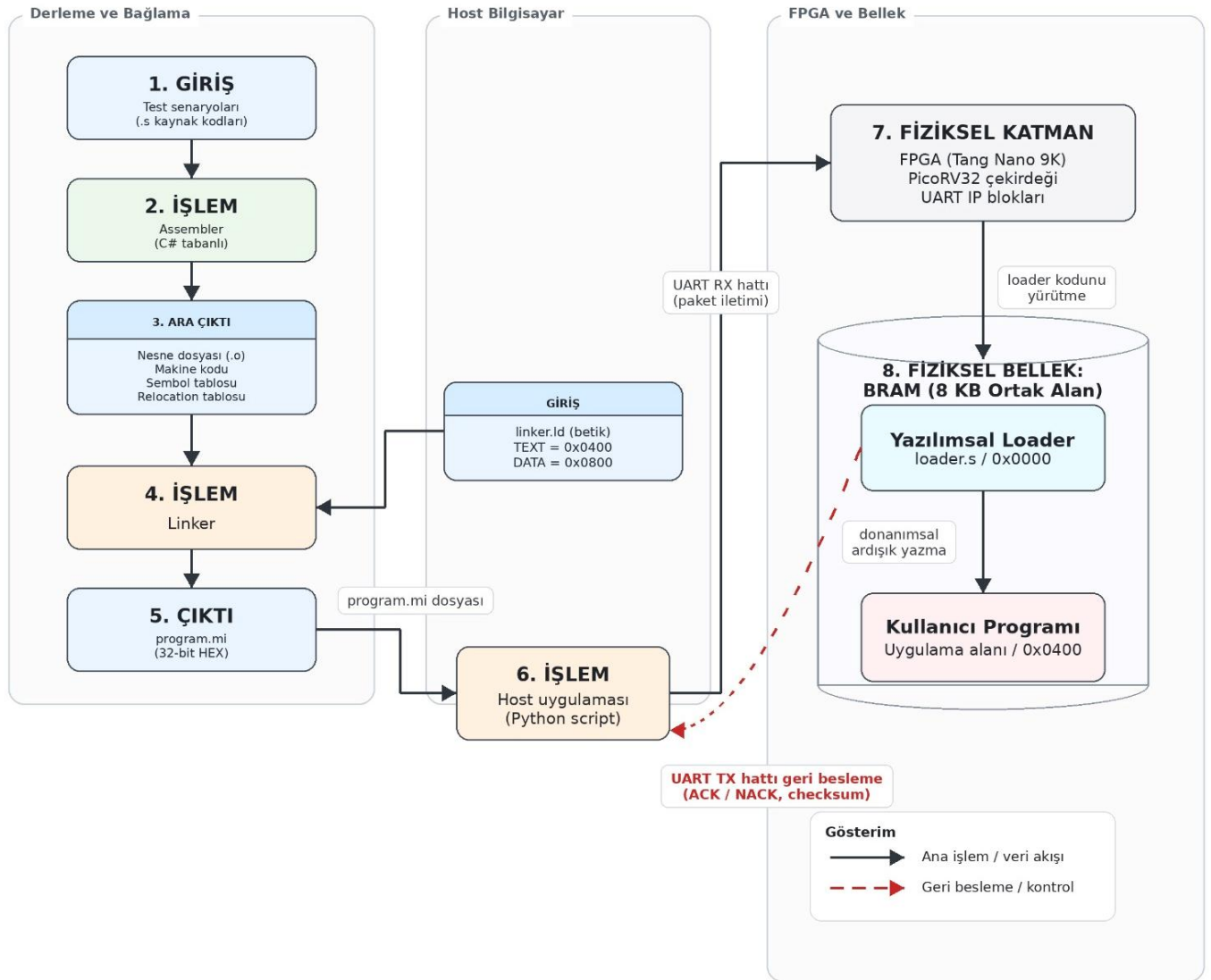
Derleme (Bilgisayar Aşaması): Yazdığımız kodlar, bilgisayardaki çevirici programlar (Assembler ve Linker) tarafından işlenerek çipin doğrudan anlayabileceği 32-bitlik bir makine koduna (program.mi) dönüştürülür ve hafızada nereye yerleşeceği planlanır [4]. Bu aşama kendi içinde iki kritik adımı barındırır:

- Nesne Dosyası (.o Çıktısı):** Bilgisayarda C# ile hazırladığımız çevirici program (Assembler), yazdığımız kodları düzenli bir veri listesi (JSON) haline getirir. Bu dosya; kodların kendisini, kullanılacak verileri ve farklı program parçalarının hafızada birbirini bulabilmesi için gereken bağlantı haritalarını içinde saklar.

- **Kodların Birleştirilmesi ve Hafızaya Yerleşimi:** Çipin hafızasında kodların nereye (örneğin 0x0400 adresine) ve verilerin nereye (0x0800 adresine) yerleşeceği sisteme basit bir kurallar listesiyle (linker.ld) bildirilir. Birleştirici program (Linker), parça parça olan bu dosyaları tek bir bütün haline getirirken, geçici adresleri hafızadaki gerçek yerlerine kaydırır. Son olarak, işlemcinin komutlar arasında atlarken (örneğin döngülere girip çıkarken) yolunu şaşırmaması için nokta atışı yönlendirme adreslerini makine kodunun içine eksiksiz bir şekilde işler.

Aktarım (Gönderim Aşaması): Bilgisayarda çalışan ana program (Host Uygulaması), hazırlanan bu makine kodunu okur ve bir haberleşme kablosu (UART RX Hattı) üzerinden küçük paketler halinde çipe iletir.

Yükleme ve Çalıştırma (Çip Aşaması): Çipin ortak hafızasının (BRAM) başlangıcında (0x0000) yer alan yükleyici program (Yazılımsal Loader), kablodan gelen veri paketlerini alır ve hafızada asıl çalışacakları bölgeye (0x0400) sırayla kaydeder. Veri aktarımında bir bozulma yoksa, bilgisayara Şekil 1’de belirtildiği gibi kırmızı ile belirtilen UART TX hattı üzerinden onay (ACK) mesajı gönderir. Yükleme işlemi hatasız tamamlandığında ise donanımın kontrolünü yeni yüklenen bu programa devrederek sistemi çalıştırır (PÇ6).



Şekil 1 Sistem Toolchain Şeması

2.2. FPGA Loader ve PicoRV32 Bellek Haritası (Memory Map)

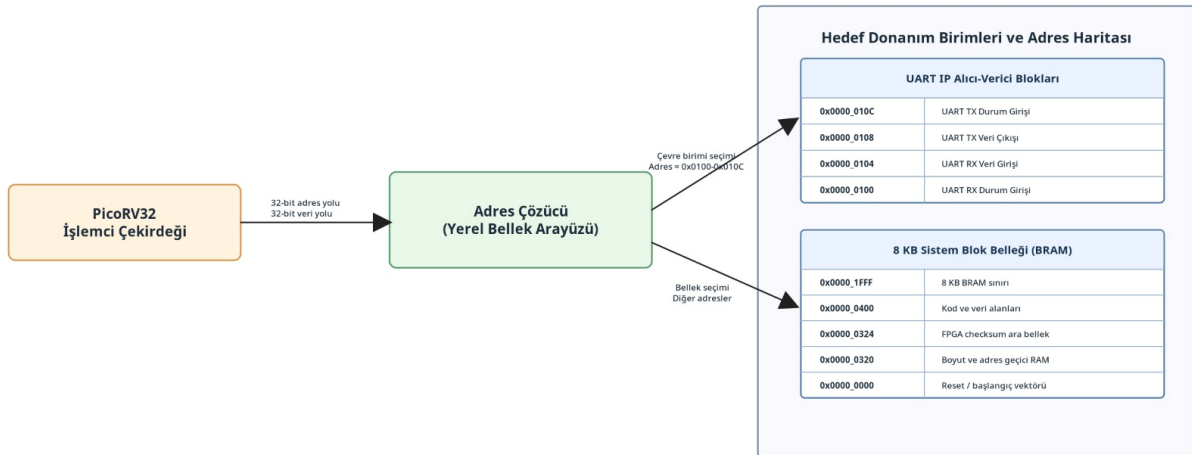
Sistem donanımında, işlemcinin ve diğer birimlerin birbiriyle tamamen uyumlu çalışabilmesi için hafıza alanı (Memory Map) belirli bölümlere ayrılmıştır. Şekil 2'de gösterilen sistem donanım mimarisi ve bellek haritasında görüldüğü gibi, 8 KB'lık ortak sistem belleği (BRAM) ve iletişim birimleri şu adres aralıklarında konfigüre edilmiştir:

0x0000_0000 (Reset / Başlangıç Vektörü): İşlemci elektrik aldığı an ilk olarak bu adrese bakar ve sistem yükleyici (Loader) kodunu buradan çalıştırmaya başlar.

0x0000_0100 - 0x0000_010C (UART Giriş/Çıkış Kapıları): Bilgisayardan gelen verilerin çip tarafından okunmasını ve çipten bilgisayara geri bildirim gönderilmesini sağlayan donanımsal iletişim adresleridir.

0x0000_0320 - 0x0000_0324 (Geçici Ara Bellek): Bilgisayardan gelen programın boyutu, hafızada nereye yazılacağı bilgisi ve hata kontrolü (Checksum) değerleri bu hücrelerde geçici olarak saklanır.

0x0000_1FFF (Donanım BRAM Sınırı): Kodların ve verilerin ortaklaşa saklandığı 8 KB'lık ana blok belleğin son sınırındır.



Şekil 2 Bellek Haritası

Yükleyici programın çalışma mantığı ve bilgisayarla olan iletişimi, doğrudan bir sonlu durum makinesi (FSM) mimarisine dayanır. Şekil 3'teki yükleyici durum geçiş diyagramında detaylandırıldığı gibi, sistem harici hiçbir donanım müdahalesine ihtiyaç duymadan şu adımları sırasıyla takip eder:

STATE_INIT (Başlangıç ve Bekleme): İşlemci açıldığı an bu duruma geçer. İletişim adresleri tanımlanır, hata kontrol sayacı sıfırlanır ve bilgisayardan ilk verinin gelmesi beklenir.

STATE_READ_SIZE (Boyut Okuma): Bilgisayardan gönderilen 4 byte'lık program boyutu okunur. Gelen veriler geçici belleğe kaydedilirken aynı zamanda hata kontrol sayacına eklenir.

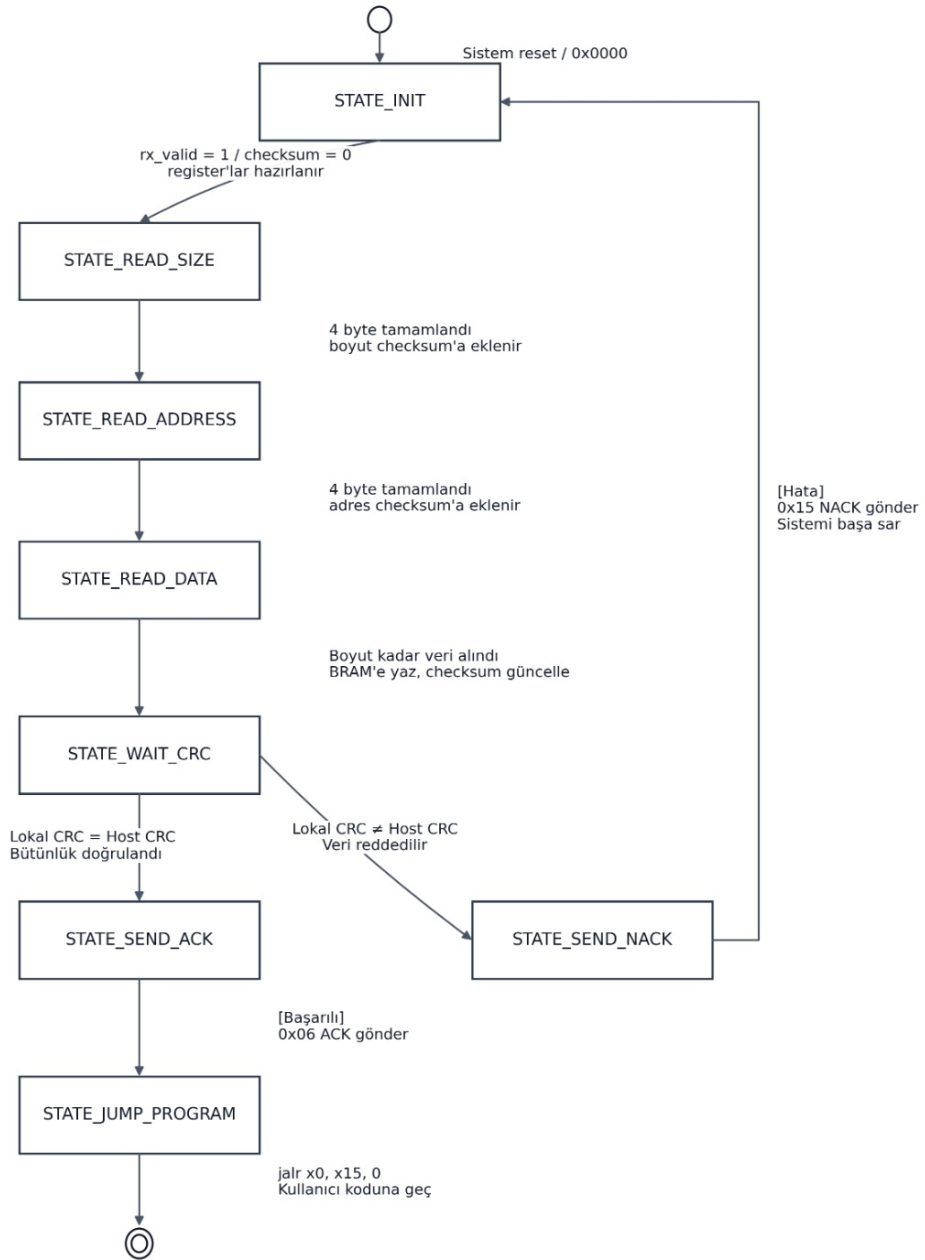
STATE_READ_ADDRESS (Hedef Adres Okuma): Programın hafızada tam olarak nereye yazılacağını belirten 4 byte'lık başlangıç adresi bilgisayardan alınır ve işlem bittiğinde oraya sıçramak üzere güvenli bir yerde saklanır.

STATE_READ_DATA (Makine Kodlarını Yazma): Bilgisayardan gelen asıl makine kodları sırayla okunarak hedef hafıza adreslerine yazılır. Gelen her yeni parça, hata kontrol sayacına dahil edilerek süreç takip edilir.

STATE_VERIFY_CHECKSUM (Hata Doğrulama): Kodların aktarımı bittiğinde, bilgisayarın hesapladığı doğrulama kodu ile çipin kendi topladığı kod karşılaştırılır. Eğer kodlar eşleşmiyorsa hata bildirimi (NACK) gönderilir ve sistem en başa dönerek işlemi sıfırlar.

STATE_JUMP / RESET (Sistemi Ayağa Kaldırma): Doğrulama başarılıysa bilgisayara "veriler hatasız ulaştı" (ACK) mesajı fırlatır. İşlemci anında yükleyici modundan çıkarılarak yeni yüklenen programın başlangıç adresine (örneğin 0x0400 adresine) yönlendirilir ve donanım canlı olarak çalıştırılmaya başlar (PÇ1).

FPGA Yükleyici (Loader) Sonlu Durum Makinesi (FSM)



Şekil 3 Loader FSM Şeması

3. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ

3.1. Deney Tasarımı ve Test Senaryoları

Sistemin doğru çalıştığını ve donanımın kararlı olduğunu kanıtlamak için, basitten karmaşığa doğru üç farklı Assembly test programı yazılmış ve çip üzerinde çalıştırılmıştır.

Test Senaryosu 1: Temel Matematik (ALU Doğrulaması)

Bu testin amacı, işlemcinin Aritmetik Lojik Biriminin (ALU) temel veri işleme ve toplama yeteneğini donanım üzerinde kontrol etmektir. Programda belirlenen iki sabit sayı addi komutlarıyla yazmaçlara (register) yüklenmiş, add komutuyla toplanmış ve elde edilen sonuç sw (Store Word) komutuyla doğrudan LED donanımına yazdırılmıştır. Bu basit toplama işlemi, donanımın veri yollarındaki aktarımın en net şekilde görülmesini sağlar. Bu testin mantığı, David Patterson ve John Hennessy'nin "Computer Organization and Design (RISC-V Edition)" kitabının 2.2 Operations of the Computer Hardware (Bilgisayar Donanımının İşlemleri) ve 4.3 Building a Datapath (Veri Yolu İnşası) alt başlıklarındaki temel doğrulama yöntemleri referans alınarak oluşturulmuştur[5].

Test 1: Temel Toplama İşlemi

test1.s - RISC-V ALU Doğrulama (İki Sayının Toplamı)

```
.text
.global start

start:
    addi x4, x0, 512    # x4 = LED Donanım Adresi (0x0200)

islem:
    addi x8, x0, -5      # İlk sayıyı belirle (Örnek: 5)
    addi x9, x0, 3       # İkinci sayıyı belirle (Örnek: 3)

    # Matematiksel İşlem: İki sayıyı topla
    add x10, x8, x9      # x10 (Sonuç) = x8 + x9

    # Sonucu LED'e yansıt
    sw x10, x4, 0        # Çıkan sonucu (8) LED donanımına yazdır

sonsuz_dongu:
    # İşlem bitti, ışıkların sönmemesi için sistemi burada beklet
    jal x0, sonsuz_dongu
```

Test Senaryosu 2: Döngüler ve Kaydırma / Walking One (Mantıksal Dallanma Testi)

Bu testin amacı, işlemcinin karar verme ve döngü (loop) kurma yeteneklerini donanım üzerinde doğrulamaktır. "Yürüyen Işık" programında, yanan LED'i sola kaydırmak için sayı sürekli kendisiyle toplanmış (add) ve sınır değere ulaştığında bne ve jal komutlarıyla güvenli bir şekilde başa dönmesi sağlanmıştır. Bu sayede işlemcinin dallanma (branch) komutlarını hatasız işlediği kanıtlanmıştır. Bu testin yapısı, David Harris ve Sarah Harris'in "Digital Design and Computer Architecture (RISC-V Edition)" kitabının 6. Bölümünde yer alan 6.4.2 Branching (Dallanma) ve 6.4.4 Getting Loopy (Döngüler) alt başlıklarındaki mantık referans alınarak tasarlanmıştır [6].

Test 2: Walking One (Yürüyen Işık)

test2.s - Karmaşık Döngüler (Walking One / Shift Register)

```
.text

.global start
start:
    addi x4, x0, 512    # x4 = LED Donanım Adresi (0x0200)
    addi x7, x0, 1      # x7 = Başlangıç maskesi (Sadece 0. LED yanık)

kaydir:
    sw x7, x4, 0        # Aktif bit maskesini LED donanımına yaz

    # Gecikme Döngüsü
    addi x5, x0, 1000

dis_bekle:
    addi x6, x0, 1000

ic_bekle:
    addi x6, x6, -1
    bne x6, x0, ic_bekle
    addi x5, x5, -1
    bne x5, x0, dis_bekle

    # Lojik İşlem: Sayıyı kendisiyle toplayarak x2 yap (Sola 1 Bit Kaydır)
    add x7, x7, x7

    # --- YENİ VE GÜVENLİ SINIR KONTROLÜ ---
    addi x8, x0, 64
    bne x7, x8, devam_et # x7, 64'e eşit DEĞİLSE "devam_et" etiketine zıpla

    # Sadece 64'e eşit olduğunda bu satıra düşer ve kendini sıfırlar
    addi x7, x0, 1

devam_et:
    # Döngüyü tekrarla
    jal x0, kaydir
```

Test Senaryosu 3 (Fonksiyon Çağrılarını ve Polis Çakarı):

Bu testin amacı, işlemcinin alt program (fonksiyon) çağırma ve dönüş adreslerini yönetme becerilerini kontrol etmektir. Program içinde ana döngüden çıkılarak bir gecikme alt programına atlanmış (jal), bu işlem bittiğinde ise programın kaybolmadan tam kaldığı yere geri dönmesi (jalr) sağlanmıştır. Böylece işlemcinin dönüş adresini x1 (ra) yazmacında güvenle saklayabildiği görülmüştür. Bu testin kod mimarisi, David Harris ve Sarah Harris'in "Digital Design and Computer Architecture (RISC-V Edition)" kitabının 6. Bölümünde yer alan 6.4.6 Procedure Calls (Prosedür Çağrılarını) alt başlığındaki standartlar referans alınarak oluşturulmuştur (PÇ7) [6].

Test 3: Polis Çakarı

test3.s - Fonksiyon Çağrılarını ve Dönüş Adresi (Polis Çakarı)

```
.text

.global start
start:
    addi x4, x0, 512    # x4 = LED Donanım Adresi

sonsuz_dongu:
    # Adım 1: İçteki iki LED'i yak (Binary: 001100 -> Decimal: 12)
    addi x7, x0, 12
    sw x7, x4, 0
    jal x1, gecikme_fonksiyonu    # FONKSİYON ÇAĞRISI (Dönüş adresi x1'e
    kaydedilir)

    # Adım 2: Dıştaki 4 LED'i yak (Binary: 110011 -> Decimal: 51)
    addi x7, x0, 51
    sw x7, x4, 0
    jal x1, gecikme_fonksiyonu    # FONKSİYON ÇAĞRISI (Tekrar alt programa git)

    jal x0, sonsuz_dongu          # Ana döngüyü başa sar

# =====
# ALT PROGRAM (SUBROUTINE): gecikme_fonksiyonu
# =====
gecikme_fonksiyonu:
    addi x5, x0, 1500

dis_bekle:
    addi x6, x0, 1500

ic_bekle:
    addi x6, x6, -1
    bne x6, x0, ic_bekle
    addi x5, x5, -1
    bne x5, x0, dis_bekle

# Fonksiyon bitti. Bizi çağırana yere (x1 içindeki adrese) geri dön!
jalr x0, x1, 0
```

3.2. Veri Toplama ve Donanım Metrikleri

Geliştirilen sistemde, bilgisayar tarafında çalışan ana haberleşme yazılımı (Host Uygulaması) FPGA ile gerçekleştirdiği tüm veri alışverişini otomatik olarak bir kayıt dosyasına (CSV) işlemektedir.

Şekil 4'teki sistem yükleme kayıtları (log) tablosunda detaylıca görüldüğü gibi, gerçekleştirilen testlerde programın byte cinsinden boyutu, yazılacağı hedef adres (0x0400) ve milisaniye (ms) seviyesindeki fiziksel yükleme süreleri anlık olarak ölçülmüştür. Bu görsel, tasarladığımız hata kontrol (Checksum) mekanizmasının gerçek donanım üzerinde başarıyla çalıştığını kanıtlamaktadır:

İletişim sırasında bir kopukluk veya zaman aşımı yaşandığında sistem bunu anında fark etmiş ve durumu "Başarısız (Zaman Aşımı/NACK)" olarak kaydederek donanımın yanlış kodla çalışmasını engellemiştir.

Veriler çipe eksiksiz ulaştığında ve hata kontrolü eşleştiğinde ise sistem "Başarılı (ACK)" onayı almış ve 52 ile 60 byte arasındaki kod paketlerini ortalama 7.5 ile 8.6 milisaniye gibi oldukça yüksek bir hızda hafızaya başarıyla yazmıştır.

	deney_sonucari.csv
1	Tarih / Saat;Test Dosyası;Kod Boyutu (Byte);Hedef Adres;Yükleme Süresi (ms);Sonuç
2	2026-06-06 19:27:45;program.mi;56;0x0400;0,00;Başarısız (Zaman Aşımı/NACK)
3	2026-06-06 19:27:59;program.mi;56;0x0400;0,00;Başarısız (Zaman Aşımı/NACK)
4	2026-06-06 19:28:32;program.mi;60;0x0400;0,00;Başarısız (Zaman Aşımı/NACK)
5	2026-06-06 19:29:21;program.mi;60;0x0400;8,63;Başarılı (ACK)
6	2026-06-06 19:30:45;program.mi;52;0x0400;8,67;Başarılı (ACK)
7	2026-06-06 19:31:26;program.mi;60;0x0400;7,52;Başarılı (ACK)
8	2026-06-06 19:32:02;program.mi;52;0x0400;0,00;Başarısız (Zaman Aşımı/NACK)
9	2026-06-06 19:32:38;program.mi;52;0x0400;7,51;Başarılı (ACK)

Şekil 4 Test Senaryoları Gerçekleşme Süreleri

FPGA Donanım Sentez Raporu ve Kaynak Tüketimi

Tasarımın FPGA çipi üzerinde ne kadar fiziksel alan kapladığını analiz etmek için Gowin EDA aracı üzerinden donanım sentezleme (Synthesis) raporu alınmıştır.

Şekil 5'te yer alan kaynak tüketimi raporunda görüldüğü üzere, geliştirilen işlemci ve yükleyici (Loader) mimarisi oldukça verimli bir yapıya sahiptir:

Mantık Hücreleri (Logic/LUT): Çip üzerindeki 8640 mantık hücresinin yalnızca 1471'i (%18) kullanılmıştır.

Yazmaçlar (Registers/FF): Toplam 6693 donanımsal yazmacın sadece 548'i (%9) tüketilmiştir.

Blok Bellek (BSRAM): Yükleyici ve kullanıcı programlarının ortak tutulduğu bellek blokları için çipin BSRAM kapasitesinin sadece %24'ü kullanılmıştır.

Giriş/Çıkış Portları (I/O Port): UART fiziksel kablo haberleşmesi ve LED çıkışları için 71 pinin yalnızca 10 tanesine (%15) ihtiyaç duyulmuştur.

Bu oldukça düşük kaynak tüketim oranları, tasarlanan sistemin çipi yormadığını ve ilerleyen aşamalarda çip içerisine yüklenebilecek çok daha karmaşık projeler (kullanıcı test senaryoları) için sistemde geniş bir fiziksel boş alan bırakıldığını kanıtlamaktadır (PÇ7)(PÇ12).

Resource	Usage	Utilization
Logic	1471 / 8640	18%
└ LUT, ALU, ROM16	1471 (1267 LUT, 204 ALU, 0 ROM16)	—
└ SSRAM (RAM16)	0	—
Register	548 / 6693	9%
└ Logic Register as Latch	0 / 6480	0%
└ Logic Register as FF	547 / 6480	9%
└ I/O Register as Latch	0 / 213	0%
└ I/O Register as FF	1 / 213	<1%
CLS	890 / 4320	21%
I/O Port	10 / 71	15%
I/O Buf	10	—
└ Input Buf	3	—
└ Output Buf	7	—
└ Inout Buf	0	—
BSRAM	1 SP 5 SDP8	24%

Şekil 5 Gowin EDA Enerji İstatistikleri

4. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ

Bu projede geliştirilen PicoRV32 tabanlı mikroişlemci mimarisi ve donanım-yazılım ortak tasarımı yükleyici (loader) mekanizması, yalnızca teknik bir başarı sunmakla kalmayıp; çevresel sürdürülebilirlik, ekonomik bağımsızlık ve toplum sağlığı gibi evrensel hedeflere de doğrudan hizmet etmektedir. Projenin Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları (SKA) bağlamındaki etkileri aşağıda analiz edilmiştir.

4.1. Sürdürülebilirlik ve Yeşil Bilişim (Green Computing)

Geliştirilen sistem, donanım kaynaklarının kullanımı açısından son derece optimize bir yapıya sahiptir. Sentez raporlarından Şekil 5'te (Resource Usage Summary) elde edilen verilere göre, işlemci çekirdeği ve yükleyici FSM, FPGA üzerindeki LUT (Mantık Kapısı) kapasitesinin yalnızca %18'ini, Register kaynaklarının ise %9'unu kullanmaktadır.

İşlemci ve çip tasarımlarında aktif olarak çalışan transistör ve mantık kapısı sayısının az olması, doğrudan güç tüketiminin (dinamik güç dağılımı) düşmesi anlamına gelir. Ayrıca, geliştirilen özel Linker (Bağlayıcı) yazılımı sayesinde bellekte gereksiz boşluklar bırakılmamış, programlar doğrudan 0x0400 adresinden itibaren en az yer kaplayacak şekilde paketlenmiştir. Bu donanım ve yazılım optimizasyonları, veri merkezlerinde veya endüstriyel sahalarda kullanılacak milyonlarca gömülü sistemin harcayacağı elektriği minimuma indirerek, SKA 7 (Erişilebilir ve Temiz Enerji) ve SKA 13 (İklim Eylemi) hedeflerine doğrudan katkı sağlamaktadır. Karbon ayak izini azaltan bu yaklaşım, Yeşil Bilişim (Green Computing) felsefesinin temelini oluşturmaktadır.

4.2. Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık

Günümüzde çip endüstrisi büyük oranda kapalı kaynaklı ve lisans ücreti talep eden mimarilerin (örneğin ARM) tekeli altındadır. Bu projede, tamamen açık kaynaklı RISC-V (PicoRV32) komut seti mimarisi kullanılmış ve bu mimariyi yönetecek olan Assembler, Linker ve Host yazılımları sıfırdan, yerli imkanlarla geliştirilmiştir.

Dışa bağımlı ticari yazılımlar (IDE ve derleyiciler) yerine, projedeki gibi özgün araç zincirlerinin kullanılması, bir ülkenin çip tasarımı ve Ar-Ge maliyetlerini milyonlarca dolar seviyesinden sıfıra indirebilir. Geliştirilen bu sistem, yerli çip ve savunma sanayii projelerinde herhangi bir lisans kısıtlamasına veya ambargoya takılmadan kullanılabilir altyapıya sahiptir. Bu durum, projenin SKA 8 (İnsana Yakışır İş ve Büyüme) ve SKA 9 (Sanayi, Yenilikçilik ve Altyapı) hedeflerine olan ekonomik katkısını ve teknolojik bağımsızlık vizyonunu kanıtlamaktadır.

4.3. Fonksiyonel Güvenlik ve Sağlık

Gömülü sistemler; tıbbi solunum cihazları, otonom araç fren sistemleri (ABS) veya savunma sanayii radarları gibi insan hayatının doğrudan söz konusu olduğu kritik alanlarda çalışmaktadır. Bu tür cihazlara yeni bir yazılım yüklenirken oluşabilecek tek bir bitlik hata (örneğin 0'ın 1 olması), cihazın çökmesine ve can kayıplarına yol açabilir.

Projede tasarlanan "Sıfır Kayıplı Gönderim Protokolü", donanım seviyesinde çalışan katı bir Sonlu Durum Makinesi (FSM) ile korunmaktadır. Python üzerinden gönderilen makine kodları, CRC (Checksum) algoritması ile doğrulanmakta ve FSM yalnızca verinin %100 doğru olduğunu teyit ederse (ACK - 0x06) işlemcinin çalışmasına izin vermektedir. Veride bir bozulma varsa sistem yazılımı reddederek (NACK - 0x15) önceki güvenli durumda kalır. Geliştirilen bu hata toleranslı ve güvenli yükleme mimarisi, kritik sistemlerdeki iş kazalarını ve tıbbi cihaz hatalarını önleme potansiyeliyle SKA 3 (Sağlık ve Kaliteli Yaşam) hedefini doğrudan desteklemektedir.

4.4. E-Atık Yönetimi ve Döngüsel Ekonomi

Elektronik atıklar (e-atık), günümüzün en büyük çevre sorunlarından biridir. Eski nesil ASIC (Uygulamaya Özel Tümleşik Devre) çiplerinde, yazılımda veya donanım mantığında bir hata tespit edildiğinde kartın tamamı sökölüp çöpe atılmak zorundadır.

Bu projede kullanılan FPGA mimarisi ve donanım üzerinden haberleşen dinamik BootROM yükleyicisi, çipin fiziksel olarak değiştirilmesine gerek kalmadan güncellenebilme yeteneğine sahiptir. İşlemcinin belleğindeki program veya görevler, seri haberleşme kablosu üzerinden saniyeler içinde baştan yazılabilir. Bu esnek mimari, elektronik donanımların kullanım ömrünü yıllarca uzatarak, cihazların kısa sürede işlevsiz kalıp hurdaya ayrılmasını önler. Yeni bir çip satın almak yerine sadece yazılımın güncellenmesiyle donanımların çöpe atılmasının engellenmesi, projenin SKA 12 (Sorumlu Tüketim ve Üretim) amacına sağladığı en somut mühendislik katkısıdır (PÇ8).



Şekil 6 SKA Tablosu

5. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI

Proje geliştirme sürecinde, "herkes kendi parçasını tek başına yapsın, en son birleştiririz" mantığı yerine; başından sonuna kadar omuz omuza çalıştığımız bir yöntem benimsedik. Takım içindeki iletişim ve kod birleştirme sürecimiz şu temellere dayanmaktadır:

Birlikte Öğrenme ve Karar Alma (Google Meet): Projedeki en büyük iletişim aracımız düzenli olarak yaptığımız Google Meet toplantıları oldu. PicoRV32 işlemci mimarisi, durum makinesi (FSM) tasarımı veya derleyici mantığı gibi yeni konuları herkesin kendi başına araştırması yerine; ekran paylaşarak, okuyarak ve hep birlikte tartışarak aynı anda öğrendik.

Fikir Aışverişı ve Ortak Çözüm: Kodları yazarken her aşamada birbirimizle fikir alışverişinde bulunduk. Bir sorunla karşılaştığımızda "Bu algoritmayı kullanalım" veya "Hayır, bunu yazılımda değil donanım tarafında çözmek daha güvenilir olur" şeklinde açıkça tartıştık. Bu samimi iletişim sayesinde, olası mantık hatalarını daha kodu yazmaya başlamadan, henüz düşünce aşamasındayken fark edip düzelttik.

Canlı Kod Birleştirme ve Sürüm Takibi: Yazılan farklı kodları (C#, Python, Verilog ve Assembly) birleştirirken "ekran başında canlı inceleme" yöntemini kullandık. Yazılan kodların birbirine bağlanması ve güncellenmesi, yine bu toplantılarda tüm takımın gözü önünde ve ortak onayıyla yapıldı. Böylece yazılım projelerinde sıkça yaşanan kod çakışmalarının ve "kim neresini yazmıştı" karmaşasının tamamen önüne geçtik (PÇ13).



Şekil 7 Takım Çalışması

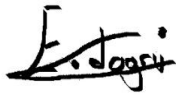
6. BİREYSEL KATKI BEYANI

Proje yönergesinde bireysel katkıların ayrı ayrı belirtilmesi istenmiştir. Ancak biz grup olarak proje sürecinde hiçbir görevi aramızda modüllere bölmedik. Donanım tasarımı, yazılım geliştirme, haberleşme protokolü ve test süreçlerinin tamamını çevrimiçi toplantılarda ekran paylaşarak birlikte yürüttük. Karşılaştığımız teknik problemleri de yine bu toplantılarda ortak kararlar alarak çözdük. Bu nedenle, projede tek bir kişinin tamamen kendi başına tasarladığı bir bölüm bulunmamaktadır. Projenin başından sonuna kadar tüm aşamaları, ekip üyelerinin eşit katılımı ve ortak çalışmasıyla tamamlanmıştır (PÇ13).

Nihat Can Akkaya



Enes Doğru



Umutcan Yıldız



Süreyya Sönmez



7. KAYNAKÇA

- [1] A. Waterman, Y. Lee, D. A. Patterson, ve K. Asanović, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1," 2016. [Online]. Available: <http://www.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.html>
- [2] W. Stallings, *Data and Computer Communications*, Pearson/Prentice Hall, 2007. [Online]. Available: https://ftp.utcluj.ro/pub/users/dadarlat/retele_an3/stuff/Data.And.Computer.Communications.8e.WilliamStallings.pdf
- [3] R. N. Williams, "A Painless Guide to CRC Error Detection Algorithms," 1993. [Online]. Available: <https://scispace.com/pdf/a-painless-guide-to-crc-error-detection-algorithms-2c9v9rqbgy.pdf>
- [4] A. V. Aho, M. S. Lam, R. Sethi, ve J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, Pearson Addison-Wesley, 2007. [Online]. Available: <https://dpvipracollege.ac.in/wp-content/uploads/2023/01/Alfred-V.-Aho-Monica-S.-Lam-Ravi-Sethi-Jeffrey-D.-Ullman-Compilers-Principles-Techniques-and-Tools-Pearson-Addison-Wesley-2007.pdf>
- [5] D. A. Patterson ve J. L. Hennessy, *Computer Organization and Design The Hardware/Software Interface: RISC-V Edition*. [Online]. Available: https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/books/HandP_RISCV.pdf
- [6] D. Harris ve S. Harris, *Digital Design and Computer Architecture*. [Online]. Available: <http://www.r-5.org/files/books/computers/hw-layers/hardware/digital-design/David Harris Sarah Harris-Digital Design and Computer Architecture-EN.pdf>

*Bilgiyi beceriyle
bütünleştiriyoruz*



Esentepe Kampüsü Serdivan/SAKARYA
T. 0 (264) 616 00 54 | F. 0 (264) 616 00 14
www.subu.edu.tr - bilgi@subu.edu.tr

